minitype-plugin-rubinate

Eito Yoneyama

Version 0.1.0

Automatic Japanese ruby for minitype. Uses Lindera with IPADIC or UniDic and furigana correspondence.

Contents

1 Quick Start

1.1 Your first ruby paragraph

Create an instance, await its autoRuby method, and place the returned inlines in a paragraph. The complete document wrapper and local installation commands are on the following reference pages and in the README.

Executable TypeScript

```typescript
import { p } from "@minitype/minitype";
import { createRubinate } from "minitype-plugin-rubinate";

const r = createRubinate();
const result = p([
  await r.autoRuby("東京スカイツリーの最寄り駅はとうきょうスカイツリー駅です。"),
]);
```

東京(とうきょう)スカイツリーの最寄(もよ)り駅(えき)はとうきょうスカイツリー駅(えき)です。

Figure 1. Your first ruby paragraph.

> p() accepts a two-dimensional inline array: [await r.autoRuby(text)] describes one line. The result is generated by the exact code above. All manual examples are executed during the build.

1.2 Using TSX

Install @minitype/tsx and set jsx to react-jsx and jsxImportSource to @minitype/tsx in tsconfig.json. AutoRuby accepts text children and RubyOptions as props and expands synchronously during JSX evaluation.

Executable TSX

```tsx
import { P } from "@minitype/tsx";
import { AutoRuby } from "minitype-plugin-rubinate/tsx";

const sample = "東京スカイツリーの最寄り駅はとうきょうスカイツリー駅です。";
const result = (
  <P>
    <AutoRuby>{sample}</AutoRuby>
  </P>
) as Block;
```

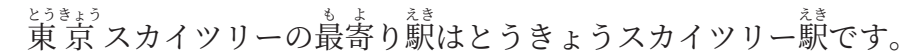

Figure 2. Using TSX.

> Pass the document directly to minitypeJSX(document).save(path). Use createAutoRuby(config) for component-wide defaults or a synchronous custom tokenizer. See examples/tsx.tsx for a complete document.

2 Usage

2.1 Correspondence and grouping

Compare correspondence lookup, tokenizer-only analysis and whole-token ruby. With correspondence, 東京 is split into 東 (とう) 京 (きょう); tokenizer-only keeps 東京 (とうきょう) together. The readings can sound identical while their ruby grouping differs.

Executable TypeScript

```ts
import { p } from "@minitype/minitype";
import { createRubinate } from "minitype-plugin-rubinate";

const r = createRubinate();
const sample = "東京スカイツリーの最寄り駅はとうきょうスカイツリー駅です。";
const result = [
  p([["Correspondence: ", ...await r.autoRuby(sample)]]),
  p([["Tokenizer only: ", ...await r.autoRuby(sample, {
    correspondence: false,
  })]]),
  p([["Whole tokens: ", ...await r.autoRuby(sample, {
    granularity: "word",
  })]]),
];
```

Correspondence: 東京(とうきょう)スカイツリーの最寄(もよ)り駅(えき)はとうきょうスカイツリー駅(えき)です。

Tokenizer only: 東京(とうきょう)スカイツリーの最寄(もよ)り駅(えき)はとうきょうスカイツリー駅(えき)です。

Whole tokens: 東京(とうきょう)スカイツリーの最寄(もよりえき)り駅はとうきょうスカイツリー駅(えき)です。

Figure 3. Correspondence and grouping.

> granularity: word includes okurigana in each annotated token. correspondence: false skips the correspondence WASM and keeps kana alignment. It does not disable morphological analysis.

2.2 Explicit readings and exclusions

Specify the reading of Akabanebashi Station explicitly, and leave Tokyo Tower unannotated. These replacements are useful for intended readings that cannot be inferred from the surrounding sentence.

Executable TypeScript

```ts
import { p } from "@minitype/minitype";
import { createRubinate } from "minitype-plugin-rubinate";

const r = createRubinate();
const sample = "東京タワーの最寄駅は赤羽橋駅です。";
const result = p([
  [...await r.autoRuby(sample)],
  [...await r.autoRuby(sample, {
    readings: { "赤羽橋駅": "あかばねばしえき", "東京タワー": null },
  })],
]);
```

東京(とうきょう)タワーの最寄(もよ)り駅(えき)は赤羽橋駅(あかはねきょうえき)です。

東京タワーの最寄(もよ)り駅(えき)は赤羽橋駅(あかばねばしえき)です。

Figure 4. Explicit readings and exclusions.

> Overrides choose the longest match at each position and bypass correspondence. Text around matches is analyzed in separate chunks, which can change context. Use userDictionary when tokenization context must be retained.

2.3 User dictionaries

Load a UTF-8 CSV file to add a word to Lindera's lattice. assets/dictionary.csv contains: 赤羽橋駅 , カスタム名詞 , アカバネバシエキ . Compare the default reading above with the corrected reading below.

Executable TypeScript

```ts
import { p } from "@minitype/minitype";
import { createRubinate } from "minitype-plugin-rubinate";
import { docsDirectory } from "../context.js";

const r = createRubinate({
  userDictionaryPath: new URL("assets/dictionary.csv", docsDirectory),
});
const sample = "東京タワーの最寄駅は赤羽橋駅です。";
const result = p([
  [...await r.autoRuby(sample, { userDictionary: "" })],
  [...await r.autoRuby(sample)],
]);
```

東京(とうきょう)タワーの最寄(もよ)り駅(えき)は赤羽橋駅(あかはねきょうえき)です。

東京(とうきょう)タワーの最寄(もよ)り駅(えき)は赤羽橋駅(あかばねばしえき)です。

Figure 5. User dictionaries.

Use a path relative to the working directory or a file URL. Files reload on each call. Specify either userDictionary or userDictionaryPath; a call-level source replaces the instance source. Missing files and invalid CSV reject the call.

2.4 Hiragana and katakana

Compare the same sentence with hiragana ruby on the first line and katakana ruby on the second. Both lines use UniDic and the same minitype styles; only the kana option changes.

Executable TypeScript

```
import { p, em, pt } from "@minitype/minitype";
import { createRubinate } from "minitype-plugin-rubinate";

const r = createRubinate({ dictionary: "unidic" });
const sample = "お茶を淹れる。";
const result = p([
  [...await r.autoRuby(sample, { kana: "hiragana" })],
  [...await r.autoRuby(sample, { kana: "katakana" })],
], {
  size: pt(12), lineHeight: em(2),
  rubySize: em(0.5), rubyOffset: em(0.1),
  rubyAlign: "jis",
});
```

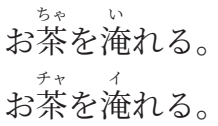

Figure 6. Hiragana and katakana.

Numeric minitype lengths are millimeters; pt() and em() express other units. Use sufficient lineHeight for the chosen ruby size and gap. Document-wide defaults belong in block.paragraph.

2.5 Instance defaults and overrides

Instance settings provide defaults for each call. A call replaces scalar options and merges reading entries by surface. The first paragraph leaves Tokyo Tower unannotated; the second supplies its reading in hiragana.

Executable TypeScript

```
import { p } from "@minitype/minitype";
import { createRubinate } from "minitype-plugin-rubinate";

const r = createRubinate({
  kana: "katakana",
  readings: { "赤羽橋駅": "あかばねばしえき", "東京タワー": null },
});
const result = [
  p([await r.autoRuby("東京タワーの最寄駅は赤羽橋駅です。")]),
  p([await r.autoRuby("東京タワーの最寄駅は赤羽橋駅です。", {
    kana: "hiragana",
    readings: { "東京タワー": "とうきょうたわー" },
  })]),
];
```

東京タワーの最寄駅(モヨリエキ)は赤羽橋駅(アカバネバシエキ)です。

東京(とうきょう)タワーの最寄駅(もよりえき)は赤羽橋駅(あかばねばしえき)です。

Figure 7. Instance defaults and overrides.

> A call-level userDictionary string replaces the instance CSV; strings are not concatenated. Top-level autoRuby(), analyze() and segments() use a shared default instance.

2.6 Analysis table

Build a table directly from analyze(). Each row shows a token, its reading, part of speech and original text range. This example uses UniDic; a dash indicates a missing value.

Executable TypeScript

```ts
import { p, table } from "@minitype/minitype";
import { createRubinate } from "minitype-plugin-rubinate";

const r = createRubinate({ dictionary: "unidic" });
const tokens = await r.analyze("お茶を淹れる。");
const rows = [
  ["Surface", "Reading", "Part of speech", "Range"],
  ...tokens.map(t => [
    t.surface, t.reading || "—",
    t.partOfSpeech?.filter(s => s !== "*").join(" / ") || "—",
    `[${t.start}, ${t.end})`,
  ]),
];
const result = table(rows.map(row => row.map(cell => ({
  type: "tableCell",
   block: p(cell, { size: 3.2, lineHeight: 5, align: "left", indent: 0, firstIndent: 0 })
}))), {
  columnWidths: [24, 30, 65, 30],
  cellPadding: { type: "physical", top: 2, bottom: 2, left: 2, right: 2 },
});
```

Surface	Reading	Part of speech	Range
お	お	接頭辞	[0, 1)
茶	ちゃ	名詞 / 普通名詞 / 一般	[1, 2)
を	を	助詞 / 格助詞	[2, 3)
淹れる	いれる	動詞 / 一般	[3, 6)
。	—	補助記号 / 句点	[6, 7)

Table 1. Analysis table.

> Offsets are UTF-16 indices with an exclusive end. text.slice(start, end) reproduces the token surface. autoRuby() renders segments directly. rubyKind is informational metadata and does not select a minitype layout mode.

2.7 Custom tokenizers

A custom callback avoids loading the bundled WASM. This intentionally narrow example supplies the sentence and its punctuation separately. Kana-anchor alignment separates the okurigana.

Executable TypeScript

```
import { p } from "@minitype/minitype";
import { createRubinate } from "minitype-plugin-rubinate";

const r = createRubinate({
  tokenizer: async (text) => {
    if (text !== "お茶を淹れる。") throw new Error("Unsupported input");
    return [
      { surface: "お茶を淹れる", reading: "オチャヲイレル" },
      { surface: "。", reading: "" },
    ];
  },
});
const result = p([await r.autoRuby("お茶を淹れる。")]);
```

お茶(ちゃ)を淹(い)れる。

Figure 8. Custom tokenizers.

> Production callbacks must preserve surface text and order. They receive a second argument with userDictionary and correspondence settings and decide how to honor them. Optional segments can provide explicit correspondence.

2.8 Validation and errors

Invalid user CSV rejects before it enters the analyzer. The message below is captured at build time, followed by a successful analysis to show that the failure does not corrupt the next request.

Executable TypeScript

```
import { p } from "@minitype/minitype";
import { autoRuby } from "minitype-plugin-rubinate";

let message = "";
try {
  await autoRuby("お茶を淹れる。", { userDictionary: "only-one-column" });
} catch (error) {
  message = error instanceof Error ? error.message : String(error);
}
const result = [
  p(message, { align: "left" }),
  p([["Next valid call: ", ...await autoRuby("お茶を淹れる。")]]),
];
```

userDictionary must be valid three-column CSV: surface,-

part-of-speech,kana-reading

Next valid call: お茶(ちゃ)を淹れる。

Figure 9. Validation and errors.

> Other failures include invalid kana or granularity options, missing assets and custom tokenizers that lose text. Catch rejected promises at your application's boundary.

2.9 Choosing UniDic

The first line uses IPADIC, which leaves 淹 unannotated in this sentence. The second uses UniDic, which supplies い. Select a dictionary on an instance or an individual call.

Executable TypeScript

```
import { p } from "@minitype/minitype";
import { createRubinate } from "minitype-plugin-rubinate";

const r = createRubinate({ dictionary: "unidic" });
const sample = "お茶を淹れる。";
const result = p([
  [...await r.autoRuby(sample, { dictionary: "ipadic" })],
  [...await r.autoRuby(sample)],
]);
```

お茶(ちゃ)を淹れる。

お茶(ちゃ)を淹(い)れる。

Figure 10. Choosing UniDic.

> The bundled dictionaries differ in vocabulary and tokenization; UniDic is not necessarily more accurate for every sentence. correspondence: false skips correspondence lookup while retaining the analyzer's reading reconstruction. Only the selected analyzer loads at runtime.

3 Public API

3.1 Functions and options

Import runtime functions and public types from minitype-plugin-rubinate. All three asynchronous functions are also methods on the object returned by createRubinate().

TypeScript reference

```
createRubinate(config?: RubinateConfig): Rubinate
autoRuby(text: string, options?: RubyOptions)
  : Promise<(string | Ruby)[]>
analyze(text: string, options?: RubyOptions)
  : Promise<AnalyzedToken[]>
segments(text: string, options?: RubyOptions)
  : Promise<Segment[]>
alignReading(surface: string, reading: string): Segment[]

type Dictionary = "ipadic" | "unidic";
interface RubyOptions {
  dictionary?: Dictionary;        // ipadic
  kana?: "hiragana" | "katakana"; // hiragana
  granularity?: "kanji" | "word"; // kanji
  correspondence?: boolean;       // true
  readings?: Readonly<Record<string, string | null>>;
  userDictionary?: string;
  userDictionaryPath?: string | URL;
}
interface TokenizerOptions {
  dictionary: Dictionary;
  userDictionary?: string;
  correspondence: boolean;
}
interface RubinateConfig extends RubyOptions {
  tokenizer?: (text: string, options: TokenizerOptions)
    => readonly Morpheme[] | Promise<readonly Morpheme[]>;
}
```

> createRubinate() returns immediately; analysis validates options and rejects on failure. alignReading() is synchronous and performs no dictionary lookup. Kana anchors must have one complete alignment; ambiguous correspondence falls back to a whole-token group.

3.2 TSX components

Import AutoRuby or createAutoRuby from minitype-plugin-rubinate/tsx. AutoRuby analyzes text synchronously during JSX evaluation and directly returns minitype ruby inlines, so minitypeJSX can typeset the document without a resolver pass.

TypeScript reference

```
AutoRuby(props: AutoRubyProps): (string | Ruby)[]
createAutoRuby(config?: SyncRubinateConfig): AutoRubyComponent

type AutoRubyText = string | number | boolean
  | null | undefined | readonly AutoRubyText[];
interface AutoRubyProps extends RubyOptions {
  children?: AutoRubyText;
}
interface SyncRubinateConfig extends RubyOptions {
  tokenizer?: (text: string, options: TokenizerOptions)
    => readonly Morpheme[];
}
```

createAutoRuby supplies component-wide defaults; each element's RubyOptions override them. Text children are concatenated, numbers become text, and boolean or nullish children are ignored. Nested JSX elements are rejected; place formatting around AutoRuby and manual ruby beside it.

> Call minitypeJSX(document) directly. TSX evaluation is synchronous, so custom tokenizers supplied to createAutoRuby must also be synchronous. The normal createRubinate API remains asynchronous and accepts synchronous or asynchronous custom tokenizers. Missing files and invalid options throw while the JSX tree is constructed.

3.3 Tokens and correspondence

TypeScript reference

```ts
interface Segment {
  text: string;
  reading?: string;
}
interface Morpheme {
  dictionary?: Dictionary;
  surface: string;
  reading: string;
  partOfSpeech?: readonly string[];
  dictionaryForm?: string;
  details?: readonly string[];
  segments?: Segment[];
  furiganaSource?: "jmdict" | "jmnedict" | "none";
  furiganaMatch?: "exact" | "inferred" | "none";
  rubyKind?: "mono" | "jukugo" | "group";
}
interface AnalyzedToken extends Morpheme {
  start: number; // UTF-16, inclusive
  end: number;   // UTF-16, exclusive
  source: "lindera" | "custom" | "override" | "gap";
  segments: Segment[];
}
```

Dictionary details

Known tokens retain the nine IPADIC fields: POS, three POS subdivisions, conjugation type, conjugation form, base form, reading and pronunciation. Reading is index 7; pronunciation is index 8. UniDic retains 17 fields; its lemma reading is index 6, surface pronunciation 9 and orthographic base 10. dictionary identifies the schema; unknown tokens may have fewer fields.

> Correspondence first checks the proper-name dictionary for proper nouns. A found correspondence can adjust the selected reading. Use correspondence: false for raw tokenizer readings; use readings for an unconditional override.

For custom tokens, segments must reconstruct surface exactly. Without segments, Rubinate applies its independent alignment. Whitespace omitted by a custom analyzer is restored; omitted non-whitespace text rejects.

3.4 Layout and compatibility

Complete document wrapper

TypeScript reference

```
import { minitype, p, em } from "@minitype/minitype";
import { autoRuby } from "minitype-plugin-rubinate";

await minitype([{ body: [
  p([await autoRuby("お茶を淹れる。")]),
] }], {
  block: { paragraph: {
    lineHeight: em(2),
    rubySize: em(0.5), rubyOffset: em(0.1),
  } },
}).save("output.pdf");
```

Analysis and reading alignment

The embedded Lindera/IPADIC and UniDic analyzers, Rust okurigana alignment and JmdictFurigana/JmnedictFurigana lookup use Rubinate's buffer ABI. Both analyzers run in normal mode. UniDic reconstructs orthographic readings from lemma and inflected pronunciation.

What minitype controls

Each annotated segment becomes an ordinary minitype ruby inline. rubyKind is informational classification from the correspondence dictionary, or a fallback inferred from the segment structure. It does not affect rendering. minitype controls placement and line breaking; Typst-specific jukugo behavior, intrusion/protrusion calculations and two-sided ruby are not reproduced.

> Pass text to autoRuby(), or use AutoRuby directly in TSX. AutoRuby expands only its own text children; surrounding text and manual ruby remain as authored. Keep paragraph boundaries in the host document. Readings for ambiguous names still require editorial review.

3.5 Build, runtime and provenance

Build this repository

Shell commands

```
npm ci
npm test
npm run example
npm run documentation
npm pack
```

After npm run build, install the local directory in your document project with npm install /path/to/minitype-plugin-rubinate and install @minitype/minitype. The package has not been published to npm.

Runtime and shipped assets

Node.js 20.16+ (20.x) or 22.3+; minitype 0.1.6+. The Node loader reads local assets, with no analysis-time network request or native addon. IPADIC is about 11 MiB, UniDic 44 MiB and correspondence 5.4 MiB. All are packaged; only selected modules load lazily and are shared per process. The async API still runs synchronous WASM work; use a worker when event-loop latency matters.

Exact source and rebuilds

assets/manifest.json records SHA-256 hashes of the shipped binaries and sources. Rust sources, Cargo locks and compressed dictionary data are maintained under wasm-plugins. Lindera 1.4.1 uses the pinned rice8y fork. The correspondence snapshot is 2.3.1+2026-05-25.

Rebuild the WASM assets

```
rustup target add wasm32-unknown-unknown
node scripts/rebuild-wasm.mjs
npm test
```

Rebuilding requires Cargo and initial network access for dependencies and dictionary archives. It is optional: shipped WASM runs as installed. Binary identity across Rust toolchains is not promised.

> License terms and component notices are summarized in the following License section.

4 License

Rubinate is distributed under the MIT License. The package embeds precompiled WASM components: morphological analysis uses Lindera under the MIT License, with IPADIC under its original dictionary terms and UniDic 2.1.2 under its BSD-3-Clause option. Rendering is handled by the peer dependency @minitype/minitype, licensed under PolyForm Noncommercial 1.0.0. The TSX examples use @minitype-/tsx under the MIT License. See LICENSE and THIRD_PARTY_NOTICES.md for the component-to-license mapping and the locations of the full license texts.

The bundled JmdictFurigana and JmnedictFurigana correspondence data are derived from EDRDG's JMdict/EDICT and JMnedict/ENAMDICT data and are distributed under CC BY-SA 4.0. See THIRD_PARTY_NOTICES.md and wasm-plugins/NOTICE.md for attribution; assets/manifest.json records the shipped source and binary hashes.